

Shortest Time to Completion First (STCF)

STCF runs the process with the least CPU work left. Unlike SJF, it can interrupt a running process when a shorter one becomes ready.

1. Model

The visualizer uses one CPU, whole ticks, and CPU-only processes. **Remaining service** is the number of CPU ticks a process still needs. It starts at the input service time and decreases by one for each tick the process runs. Switching processes takes no time.

A process can run only after it arrives. Work already completed is preserved when the process is interrupted.

2. Scheduling rules

Rule 1. Select the shortest remaining service

When the CPU is free, compare all ready processes and choose the one with the least remaining service. Use what remains now, not the original service time.

For example, A may have started with ten ticks but have only two left. If B still needs three, A is the shorter choice.

Rule 2. Preempt only for a strictly shorter process

If a ready process has less remaining service than the current process, interrupt the current process and select the shortest ready alternative. If their remaining times are equal, keep the current process running.

If A has three ticks left and B arrives needing three, A continues. If B needs two instead, B preempts A. There is no quantum that forces processes to take turns.

Rule 3. Preserve work when a process is preempted

Append the interrupted process to the ready-queue tail without changing its remaining service. When selected again, it resumes from that amount.

The queue is displayed in insertion order. Selection still searches the whole queue, so returning to the tail does not mean waiting for every process ahead of it.

Rule 4. Break dispatch ties by arrival, then input order

When the CPU is free and several processes share the shortest remaining time, choose the earlier arrival. If arrivals also match, choose the one listed first in the input.

This tie rule applies to dispatch. It does not displace an equally short process that is already running; Rule 2 keeps that process on the CPU.

Rule 5. Complete at zero; idle only if nothing is ready

Remove a process when its remaining service reaches zero. Another process may start at that same boundary. If none is ready, wait for the next arrival. Stop after all processes finish.

3. Events at a tick boundary

After accounting for the preceding tick:

1. Remove the running process if it has finished.
2. Admit all arrivals at the current time, in input order.
3. If a process is still running, check whether any ready process is strictly shorter. If so, append the interrupted process to the ready queue.
4. If the CPU is free, select the shortest remaining service, using Rule 4 for ties.

All simultaneous arrivals are considered before selecting a process. New arrivals enter the queue before an interrupted process is appended.

4. Worked example

Input order is A, B, C. All times are in ticks.

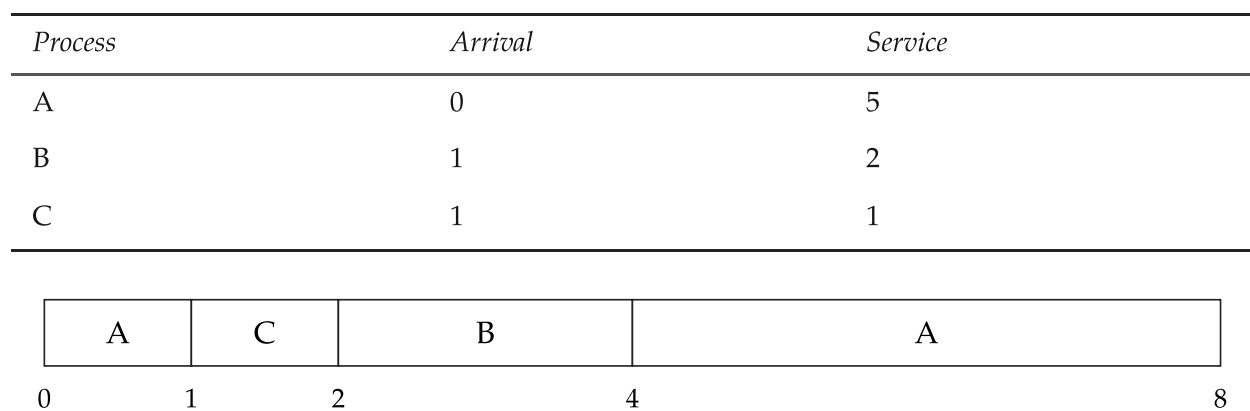


Figure 1. CPU schedule (time in ticks).

A starts at time 0. At time 1, it has four ticks left. Both B and C arrive, and C is shortest with one tick. C preempts A and finishes at time 2. B then runs for two ticks. A resumes at time 4 and uses its remaining four ticks, completing at time 8.

Notice that B does not get a turn merely because it appears before C in the input. Input order matters only after remaining time and arrival time are tied.